



Action and State Based Computation Tree Measurement Language and Algorithms

Yaping Jing^{1(✉)} and Andrew S. Miner²

¹ Salisbury University, Salisbury, MD, USA

yxjing@salisbury.edu

² Iowa State University, Ames, IA, USA

asminer@iastate.edu

Abstract. The computation tree measurement language (CTML) is a real-valued specification formalism, designed to express performance measures and dependability properties in a single framework. It is a further extension of probabilistic model checking logic in such a way that it expands the input and output value from the range of $[0, 1]$ to the range of $[0, \infty)$. Unlike probabilistic computation tree logic (PCTL) or continuous stochastic logic (CSL), CTML can *nest* real values. As such, it turns out that CTML can express a nontrivial subset of probabilistic linear time logic (PLTL) formulas that cannot be expressed by PCTL while keeping the overall computational complexity being polynomial in the size of both an input formula and model. Moreover, it can express queries such as “when a message is sent, what is the expected time until it is received?” that cannot be expressed by an existing probabilistic logic, because they are “probabilistic” at most. While powerful, CTML is a state-based specification formalism. In this work, we introduce a stronger formal query language, called *action and state based computation tree measurement language (asCTML)*. asCTML extends from CTML, for answering performance queries that operates over paths featured by a combination of states and actions. Inspired by the action and state based continuous stochastic logic (asCSL), asCTML supports multiple actions.

Keywords: Markov chain · Performance measures
Probabilistic model checking

1 Introduction

Previously, we introduced the *computation tree measurement language* (CTML) [1] that is designed to express performance measures and dependability properties in a single framework. It takes as inputs a probabilistic model structure with real-valued “rewards” functions [2] and a real-valued specification formalism and outputs a real value. CTML differentiates from probabilistic model checking logics in such a way that it takes a real value in the range of $[0, \infty)$

© Springer Nature Switzerland AG 2018

A. McIver and A. Horvath (Eds.): QEST 2018, LNCS 11024, pp. 190–206, 2018.

https://doi.org/10.1007/978-3-319-99154-2_12

as input, and output a real value in the range of $[0, \infty)$. As such, it can *nest* real values, in a sense that it does not require to convert a quantitative result from an intermediate formula to 0/1 (or true/false) value before an outer-level formula is being solved. Comparing to the probabilistic model checking formalisms, CTML has at least one of the following advantages: (1) It covers PCTL [3]. (2) It can express a nontrivial subset of PLTL [4] formulas that cannot be expressed by PCTL while keeping the overall computational complexity being polynomial in both the size of an input formula and model. (3) It can express queries such as “when a message is sent, what is the expected time until it is received?”; this type of queries cannot be expressed by an existing probabilistic specification formalism, because they are “probabilistic” at most.

While powerful, CTML is a state-based specification formalism. For action driven performance queries, it is either not intuitive or simply not expressible, particularly when considering a model with multiple actions between the same pair of states. Inspired by asCSL [5], in this work, we introduce a new formalism, called *action and state based computation tree measurement language* (asCTML), that extends from CTML, in support of performance queries that operate over a combination of states and actions. As such, the expressive power expands significantly. Take the classic dining philosophers model [6] as an example, with asCTML, it is natural to express survivability queries such as: “given a philosopher is *hungry*, how much amount of *food* that the philosopher consumes on average before he/she eventually *releases* the forks?”, where *food* is a state formula representing rate rewards, *hungry* and *release* are action formulas representing impulse rewards (i.e., a real value assigned to an action received at its corresponding state) [2, 7].

The remainder of the paper is organized as follows. Section 2 presents foundations for asCTML measures. Section 3 presents the syntax and semantics of the proposed language. Section 4 presents a translation algorithm from asCTML to CTML. Section 5 presents an application example and experimental results. Section 6 discusses related work. Section 7 concludes the paper and gives some directions for future work.

2 Foundations

In this section, we recall some well-known definitions, introduce notation, and define the underlying structures necessary for this work.

A (time-homogeneous) discrete-time Markov chain (DTMC) [8] is a tuple $(\mathcal{S}, \mathbf{P}, \pi_0)$, where $\mathcal{S} = \{0, \dots, N\}$ is a finite set of states with $N \in \mathbb{N}$, $\pi_0 : \mathcal{S} \rightarrow [0, 1]$ is an initial probability distribution under the constraint $\sum_{i \in \mathcal{S}} \pi_0(i) = 1$, and $\mathbf{P} : \mathcal{S} \times \mathcal{S} \rightarrow [0, 1]$ is a stochastic matrix.

Definition 1 (MAMC structure). *A DTMC with multiple actions, or MAMC for short, is a tuple $(\mathcal{S}, ACT, \gamma, \delta, \pi_0)$, where*

- $\mathcal{S}$ is a finite set of states.
- ACT is a finite set of action symbols.

- $\gamma : \mathcal{S} \times \mathcal{ACT} \rightarrow \mathcal{S}$ specifies the state transitions.
- $\delta : \mathcal{S} \times \mathcal{ACT} \rightarrow [0, 1]$ specifies the transition probabilities, under the constraint that $\forall s \in \mathcal{S}, \delta(s, \mathcal{ACT}) = \sum_{a \in \mathcal{ACT}} \delta(s, a) = 1$.
- $\pi_0 : \mathcal{S} \rightarrow [0, 1]$ is an initial probability distribution with $\sum_{s \in \mathcal{S}} \pi_0[s] = 1$.

Note that by the definition of γ , if $(s, a) = (s, b)$, then $\gamma(s, a) = \gamma(s, b)$ and $t = t'$, where t, t' are the next states that a, b lead to, respectively. That is, given a state s , different next states implies different actions associated with s .

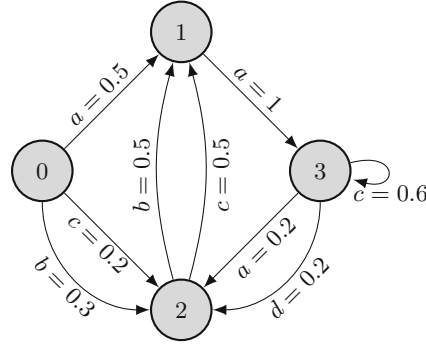


Fig. 1. An example of MAMC structure

Figure 1 shows an example of MAMC structure, with $\mathcal{S} = \{0, 1, 2, 3\}$, $\mathcal{ACT} = \{a, b, c, d\}$; at state 0, action a occurs, leading to state 1, with probability 0.5; when action b occurs, leading to state 2, with probability 0.3; when action c occurs, leading to state 2, with probability 0.2, so on and so forth.

Definition 2 (path in MAMC). Let $Y = (\mathcal{S}, \mathcal{ACT}, \Gamma, \pi_0)$ be a MAMC structure, a path in Y is an infinite sequence $\pi = (s_0, a_0), (s_1, a_1), \dots \in (\mathcal{S} \times \mathcal{ACT})^\omega$, with $\pi_i = (s_i, a_i)$, $\delta(s_i, a_i) > 0$, $s_{i+1} = \gamma(s_i, a_i)$, and $i \in \mathbb{N}$.

Definition 3 (path formula). Given a MAMC structure Y , a path formula is a function $\psi : (\mathcal{S} \times \mathcal{ACT})^\omega \rightarrow \mathbb{R}^*$, where $\mathbb{R}^*$ denotes nonnegative reals.

Definition 4 (prefix). A prefix in an MAMC structure Y is a finite sequence of state, action pairs

$$\mathbf{p} = (s_0, a_0), \dots, (s_{n-1}, a_{n-1}) \in (\mathcal{S} \times \mathcal{ACT})^n,$$

or an infinite sequence $\mathbf{p} = (s_0, a_0), (s_1, a_1), \dots \in (\mathcal{S} \times \mathcal{ACT})^\omega$, where $\mathbf{p}_i$ is the i^{th} element in $\mathbf{p}$, and $|\mathbf{p}| = n \in \mathbb{N} \cup \{\omega\}$ is the length of the sequence.

Let Ω^ω denote a set of paths in MAMC. For a given prefix $\mathbf{p}$, define $\Omega_{\mathbf{p}}^\omega$ as the set of all infinite length paths that start with prefix $\mathbf{p}$. If $|\mathbf{p}| = n \in \mathbb{N}$, we have

$$\Omega_{\mathbf{p}}^\omega = (s_0, a_0) \times \dots \times (s_{n-1}, a_{n-1}) \times \Omega^\omega \quad (1)$$

otherwise, if $|\mathbf{p}| = \omega$, then we have $\Omega_{\mathbf{p}}^\omega = \{\mathbf{p}\}$.

Definition 5 (determines ψ on MAMC). We say a prefix $\mathbf{p}$ determines ψ if, for any paths $\mathbf{x}, \mathbf{x}' \in \Omega_{\mathbf{p}}^{\omega}$, $\psi(\mathbf{x}) = \psi(\mathbf{x}')$; since all paths must have the same value for ψ in this case, we denote this quantity as $\psi(\mathbf{p})$. Note that any infinite prefix determines ψ .

Definition 6 (finitely measurable ψ on MAMC). We say a path formula ψ is finitely measurable on a MAMC structure Y if, for every path $\mathbf{x} \in \Omega_{\mathbf{p}}^{\omega}$ with $\psi(\mathbf{x}) > 0$, either there exists a finite prefix $\mathbf{p}$ with $\mathbf{x} \in \Omega_{\mathbf{p}}^{\omega}$ that determines ψ , or the probability measure for path $\mathbf{x}$ is zero.

Like CTML, we wish to define a measure of the expected value of a path formula ψ . Let $\mathcal{X}$ be any set. A σ -algebra is a subset $\mathcal{G}$ of the power set of a set $\mathcal{X}$ such that:

1. $\mathcal{X}, \emptyset \in \mathcal{G}$.
2. if $\mathcal{C} \in \mathcal{G}$, then $\mathcal{X} \setminus \mathcal{C} \in \mathcal{G}$. That is, $\mathcal{G}$ is closed under complementation.
3. if $\mathcal{C}_0, \mathcal{C}_1, \dots$ with $\mathcal{C}_i \in \mathcal{G}$, then $\bigcup_{i=0}^{\infty} \mathcal{C}_i \in \mathcal{G}$.

An important property is that, for any collection $\mathcal{C}$ of subsets of $\mathcal{X}$, there is a unique smallest σ -algebra on $\mathcal{X}$ that includes (i.e., is a superset of) $\mathcal{C}$, called the σ -algebra *generated by* $\mathcal{C}$. Then, a function $\mu : \mathcal{G} \rightarrow [0, \infty]$ is a *measure* if $\mu(\emptyset) = 0$, and for any sequence $\mathcal{C}_0, \mathcal{C}_1, \dots$ of disjoint sets in $\mathcal{G}$,

$$\mu\left(\bigcup_{i=0}^{\infty} \mathcal{C}_i\right) = \sum_{i=0}^{\infty} \mu(\mathcal{C}_i). \quad (2)$$

A more rigorous treatment of measure theory may be found for example in [9].

Definition 7 (measure of the expected value of ψ on MAMC). For any finitely measurable formula ψ on $Y = (\mathcal{S}, \mathcal{ACT}, \gamma, \delta, \pi_0)$, we define the measure of the expected value $\mu_{\psi} : \mathcal{G}_{\mathcal{S} \times \mathcal{ACT}} \rightarrow \mathbb{R}^*$ by

$$\mu_{\psi}(\Omega_{\mathbf{p}}^{\omega}) = \begin{cases} \psi(\mathbf{p}) \prod_{i=1}^{|\mathbf{p}|-1} \delta(s_i, a_i) & \text{if } \mathbf{p} = ((s_0, a_0), (s_1, a_1), \dots) \text{ determines } \psi \\ \sum_{(s,a) \in \mathcal{S} \times \mathcal{ACT}} \mu_{\psi}(\Omega_{(\mathbf{p}, (s,a))}^{\omega}) & \text{otherwise} \end{cases}$$

with $\mu_{\psi}(\emptyset) = 0$.

Note that in this measure definition of the extended model of MAMC, since the starting point is now a pair of state, action, the probability of the initial action associated with (s_0, a_0) is not included, rather, it is treated as part of the initial distribution and captured at the end.

Similar to CTML, for this work, every path formula ψ is finitely measurable. The finitely measurable property helps us to avoid the difficulty of having to observe an infinitely-long prefix to determine the value of ψ , since any such path with $\mathbf{p} = \pi$ must have $\mu_{\psi}(\Omega_{\pi}^{\omega}) = 0$.

3 Action and State Based Computation Tree Measurement Language

In this section, we introduce a new formal language called *Action and state based computation measurement language*, or asCTML for short. asCTML is designed to express performance queries based on action and/or state properties over probabilistic systems. asCTML is syntactically the same, but semantically more powerful than CTML, for supporting multiple actions, and paths featured by a combination of states and actions. The main differences between asCTML and CTML are in their semantics, the interpretation structure, the underlying paths, and the basic set of atomic functions. The new language is largely inspired by asCSL [5] and CSL^{TA} [10], for reasoning about both state and action properties over probabilistic systems. Unlike asCSL and CSL^{TA}, however, this language works naturally with the performance measures of *reward* functions [2, 7, 11] that are used to be specified within high level models. In the following, we first give the definition of the basic formulas. Then, we present syntax and semantics of the languages.

Definition 8 (state+action formula). *A state+action formula φ is defined as a function that maps from a pair of state and action to a nonnegative real value.*

$$\varphi : \mathcal{S} \times \mathcal{ACT} \rightarrow \mathbb{R}^*$$

Definition 9 (restricted state+action formula). *A restricted state+action formula φ_r is defined as a function that maps from a pair of state and action to a real value in interval $[0, 1]$.*

$$\varphi_r : \mathcal{S} \times \mathcal{ACT} \rightarrow [0, 1]$$

In principle, a state+action formula f ranges over $\mathcal{S}$ and $\mathcal{ACT}$, meaning they are defined on every pair of $(s, a) \in \mathcal{S} \times \mathcal{ACT}$; in practice, however, only the pairs with positive value of f are given explicitly. To this end, we assume the following sets of atomic items that are required for the specification of asCTML, but not required to be tied to a model.

- a finite set $\mathcal{AF}$ of atomic *state+action* formulas,
- $\mathcal{AR} \subseteq \mathcal{AF}$ of restricted atomic *state+action* formulas.

For convenience, we also assume $one, zero \in \mathcal{AR}$, that evaluate to 1 and 0, respectively, over any element $(s, a) \in (\mathcal{S} \times \mathcal{ACT})$.

3.1 asCTML Syntax

Given the definition of atomic *state+action* formula, the syntax of asCTML and the meanings of operators M (“expected value” of a path formula), X (“next”), U (“until”), and V (“weak until”) are kept the same as CTML, except:

- asCTML operates on state+action formulas, not merely state formulas.
- asCTML supports paths of combination of states and actions, whereas CTML supports paths of sequences of states only.
- asCTML distinguishes among multiple actions, whereas CTML does not.

Definition 10 (syntax of asCTML). *Let f be a state+action formula, let r be a restricted state+action formula, and let ψ_r be a restricted path formula defined as $\psi_r : (\mathcal{S} \times \mathcal{ACT})^\omega \rightarrow [0, 1]$, the syntax of asCTML can be recursively defined as follows:*

$$\begin{aligned} \varphi &::= f \mid \varphi_r \mid \varphi \odot \varphi \mid M\psi \\ \varphi_r &::= r \mid 1 - \varphi_r \mid \varphi \bowtie \varphi \mid \varphi_r \times \varphi_r \mid M\psi_r \\ \psi &::= X\varphi \mid \varphi U_{\odot}^{\leq t} \varphi \mid \varphi V_{\odot}^{\leq t} \varphi \mid \varphi U_+ \varphi \mid \varphi_r U_{\times} \varphi \mid \varphi_r V_{\times} \varphi \\ \psi_r &::= X\varphi_r \mid \varphi_r U_{\odot}^{\leq t} \varphi_r \mid \varphi_r V_{\odot}^{\leq t} \varphi_r \mid \varphi_r U_{\times} \varphi_r \mid \varphi_r V_{\times} \varphi_r \end{aligned}$$

where $\bowtie \in \{\leq, \geq, <, >\}$, $\odot \in \{+, \times\}$, $f \in \mathcal{AF}$, and $f_r \in \mathcal{AR}$.

Finally we note that asCTML's top level formula is a *state+action* formula. Also, for the same rationale as CTML, the unbounded operators $\varphi_r U_{\times} \varphi$ and $\varphi_r V_{\times} \varphi$ require that φ_r is restricted to values no greater than one.

3.2 Semantics of asCTML

We now give the formal semantics of the operators appearing in the language. Unlike model checking logics such as PCTL or asCSL, which define a satisfaction relation, asCTML formulas are defined as real-valued functions.

Definition 11 (Semantics of asCTML). *Semantics of asCTML are inductively defined as follows:*

- If $\varphi = f$, then $\varphi(s, a) = f(s, a)$, $\forall f \in \mathcal{AF}$.
- If $\varphi = \varphi_1 \times \varphi_2$, then $\varphi(s, a) = \varphi_1(s, a) \times \varphi_2(s, a)$.
- If $\varphi = \varphi_1 + \varphi_2$, then $\varphi(s, a) = \varphi_1(s, a) + \varphi_2(s, a)$.
- If $\varphi = \varphi_1 \bowtie \varphi_2$, with $\bowtie \in \{>, \geq, <, \leq\}$ then $\varphi(s, a) = 1$ if $\varphi_1(s, a) \bowtie \varphi_2(s, a)$ holds; $\varphi(s, a) = 0$ otherwise.
- If $\varphi = 1 - \varphi_1$, then $\varphi(s, a) = 1 - \varphi_1(s, a)$.
- If $\varphi = M\psi$, then $\varphi(s, a) = \mu_\psi(\Omega_{(s,a)}^\omega)$.
- If $\psi = X\varphi$, then $\psi(\pi_0, \pi_1, \dots) = \varphi(\pi_1)$.
- If $\psi = \varphi_1 U_{\odot}^{\leq t} \varphi_2$, then
 - if $\exists j \leq t$, $\varphi_2(\pi_j) > 0$, and $\forall i < j$, $\varphi_2(\pi_i) = 0$,

$$\psi(\pi_0, \pi_1, \dots) = \left(\bigodot_{i=0}^{j-1} \varphi_1(\pi_i) \right) \cdot \varphi_2(\pi_j)$$

- otherwise, $\psi(\pi_0, \pi_1, \dots) = 0$.

Also, we have $\varphi_1 U_{\odot}^{\leq \infty} \varphi_2 \equiv \varphi_1 U_{\odot} \varphi_2$.

- If $\psi = \varphi_1 V_{\odot}^{\leq t} \varphi_2$, then

- if $\exists j \leq t$, $\varphi_2(\pi_j) > 0$, and $\forall i < j$, $\varphi_2(\pi_i) = 0$,

$$\psi(\pi_0, \pi_1, \dots) = \left(\bigodot_{i=0}^{j-1} \varphi_1(\pi_i) \right) \cdot \varphi_2(\pi_j)$$

- otherwise, if $\forall i \leq t$, $\varphi_2(i) = 0$ then

$$\psi(\pi_0, \pi_1, \dots) = \bigodot_{i=0}^t \varphi_1(\pi_i)$$

Also, we have $\varphi_1 V_{\times}^{\leq \infty} \varphi_2 \equiv \varphi_1 V_{\times} \varphi_2$.

Note that the finitely measurable property of asCTML path formula ψ can be proved exactly the same CTML [1].

Finally, we define the measure of the top-level formula φ on MAMC structure M based on the initial distribution as follow.

Definition 12 (measure of M_{φ}). Given a MAMC structure $M = (\mathcal{S}, \mathcal{ACT}, \gamma, \delta, \pi_0)$, the total value of a state+action formula φ on M is defined by

$$M_{\varphi} = \sum_{s \in \mathcal{S}} \left(\sum_{a \in \mathcal{ACT}} \varphi(s, a) \cdot \delta(s, a) \right) \cdot \pi_0(s). \quad (3)$$

4 asCTML Computation

The computation of asCTML can be reduced to the computation of CTML [1] via a translation algorithm described below. The algorithm takes a MAMC structure and a set of asCTML formula, and produces a DTMC structure and a corresponding set of CTML formula. It is then proved that the translated DTMC+CTML formula always gives the same value as the MAMC+asCTML formula.

MAMC+asCTML to DTMC+CTML Translation Algorithm:

- **MAMC to DTMC:** Given a MAMC structure $M = (\mathcal{S}, \mathcal{ACT}, \gamma, \delta, \pi_0)$, a corresponding DTMC $D = (\mathcal{S}', \mathbf{P}, \pi'_0)$ can be built by the following:
 - $\mathcal{S}' = \{(sa) | (sa) \in \mathcal{S} \times \mathcal{ACT}, \delta(s, a) > 0\}$.
 - For all $(sa), (s'a') \in \mathcal{S}'$,

$$\mathbf{P}[(sa), (s'a')] = \begin{cases} \delta(s', a') & \text{if } \gamma(s, a) = s', \\ 0 & \text{otherwise.} \end{cases}$$

- $\pi'_0[(sa)] = \pi_0[s] \cdot \delta(s, a)$.

Given this model translation, it is clear that the number of states in the translated DTMC equals to the number of actions (arcs) in MAMC. The number of arcs in the translated DTMC is bounded by the number of actions in MAMC multiplied by the largest degree of outgoing arcs of a certain state.

– **asCTML to CTML:**

- Given a set of atomic state+action formulas $\mathcal{AF}$, for each $f \in \mathcal{AF}$ on M , we have $f' \in \mathcal{AF}'$ on D , with $f(s, a) = f'(sa)$, for all $(sa) \in \mathcal{S}'$.
Note that since the two sets $\mathcal{AF}$ and $\mathcal{AF}'$ are exactly the same except one is defined on M with the parameter (s, a) being a pair of state and action and the other is defined on D with parameter (sa) being a state, for the following discussion, we use notations such as f^M and f^D , respectively, to refer to the same formulas on different model structures.
- Any asCTML formula ϕ on M , denoted by ϕ^M , is translated to the same formula ϕ on D , denoted by ϕ^D , such that for each atomic state+action formula f^M in ϕ^M , it is replaced by the corresponding f^D in ϕ^D .

Theorem 1 (asCTML reduction to CTML). *Let ϕ^M be an asCTML formula on MAMC structure M , and D be the translated DTMC structure, then $\phi^M(s, a)$ evaluate the same as $\phi^D(sa)$ on D , where ϕ^D is the translation of ϕ^M .*

Proof. According to the translation method, there is a one to one mapping between state+action pair (s, a) on M and state (sa) on D , and each atomic state+action formula f on M is mapped to an atomic state formula f on D . As such, $f^M = f^D$ holds trivially. By structural induction, assume that $\phi^M = \phi^D$, $\phi_1^M = \phi_1^D$, and $\phi_2^M = \phi_2^D$. Then,

- $(\phi_1 \times \phi_2)^M = (\phi_1 \times \phi_2)^D$, $(\phi_1 + \phi_2)^M = (\phi_1 + \phi_2)^D$, $(\phi_1 \bowtie \phi_2)^M = (\phi_1 \bowtie \phi_2)^D$, and $(1 - \phi)^M = (1 - \phi)^D$ hold trivially, by the assumption.

Note that for succinctness, when the context is clear (e.g., when the parameter is given as a state such as (sa) , then it is a CTML formula; if the parameter is given as a pair of state and action such as (s, a) , then it is an asCTML formula), we drop the superscript D and M , respectively.

Also, for the following proof of $M\psi$, we utilize the fact that there exists a one to one mapping between each state, action pair (s, a) on MAMC and the translated state (sa) on DTMC. Then, each element (s_i, a_i) in a given MAMC path $\pi^M = (s, a), (s_1, a_1), \dots$ matches the state $(s_i a_i)$ in the corresponding path $\pi^D = sa, s_1 a_1, \dots$ of the translated DTMC.

- $(MX\phi)^M = (MX\phi)^D$. By asCTML semantics on $MX\phi$, for all paths: $\pi = (s, a), (s_1, a_1), \dots$, we have:

$$(MX\phi)^M(s, a) = \sum_{(s_1, a_1) \in \{\gamma(s, a)\} \times \mathcal{ACT}} \phi(s_1, a_1) \delta(s_1, a_1)$$

$$\begin{aligned} & \text{By the assumption, and translations given } s_1 = \gamma(s, a), \\ &= \sum_{(s_1 a_1) \in \mathcal{S}'} \phi(s_1 a_1) \cdot \mathbf{P}[sa, s_1 a_1] \\ &= (MX\phi)^D(sa). \end{aligned}$$

- $(M\phi_1 U_{\odot}^{\leq t} \phi_2)^M = (M\phi_1 U_{\odot}^{\leq t} \phi_2)^D$. By asCTML semantics on $M\phi_1 U_{\odot}^{\leq t} \phi_2$, for all paths of $\pi = (s, a), (s_1, a_1), \dots$, with $(s, a) = (s_0, a_0)$, if $\exists j : 0 \leq j \leq t$, s.t. $\phi_2(s_j, a_j) > 0$ and for all $0 \leq i \leq j$, $\phi_2(s_i, a_i) = 0$, we have:

$$\begin{aligned} & (M\phi_1 U_{\odot}^{\leq t} \phi_2)^M(s, a) \\ &= \sum_{j=0}^t \sum_{\substack{((s_0, a_0), \dots, (s_j, a_j)) \in (\mathcal{S} \times \mathcal{ACT})^j, \\ s_{i+1} = \gamma(s_i, a_i)}} \left(\bigodot_{i=0}^{j-1} \phi_1(s_i, a_i) \right) \cdot \phi_2(s_j, a_j) \cdot \prod_{i=1}^j \delta(s_i, a_i) \end{aligned}$$

By the assumption, and translations given $s_i = \gamma(s_{i-1}, a_{i-1})$,

$$\begin{aligned} &= \sum_{j=0}^t \sum_{(s_0 a_0, \dots, s_j a_j) \in S'^j} \left(\bigodot_{i=0}^{j-1} \phi_1(s_i a_i) \right) \cdot \phi_2(s_j a_j) \cdot \prod_{i=1}^j \mathbf{P}[s_{i-1} a_{i-1}, s_i a_i] \\ &= (M\phi_1 U_{\odot}^{\leq t} \phi_2)^D(sa). \end{aligned}$$

- $(M\phi_1 V_{\odot}^{\leq t} \phi_2)^M = (M\phi_1 V_{\odot}^{\leq t} \phi_2)^D$. According to the asCTML and CTML semantics of $M\phi_1 V_{\odot}^{\leq t} \phi_2$, there are two cases for this formula. One case is that if $\exists j : 0 \leq j \leq t$, s.t. $\phi_2(s_j, a_j) > 0$ and for all $0 \leq i \leq j$, $\phi_2(s_i, a_i) = 0$. In this case, we have just established that $(M\phi_1 U_{\odot}^{\leq t} \phi_2)^M = (M\phi_1 U_{\odot}^{\leq t} \phi_2)^D$. The second case is when $\forall i : i \leq t, \phi_2(s_i, a_i) = 0$. In this case, we have:

$$\begin{aligned} & (M\phi_1 V_{\odot}^{\leq t} \phi_2)^M(s, a) \\ &= \sum_{\substack{((s_0, a_0), \dots, (s_t, a_t)) \in (\mathcal{S} \times \mathcal{ACT})^t \\ s_{i+1} = \gamma(s_i, a_i)}} \left(\bigodot_{i=0}^t \phi_1(s_i, a_i) \right) \cdot \prod_{i=1}^t \delta(s_i, a_i) \end{aligned}$$

By the assumption, and translations given $s_i = \gamma(s_{i-1}, a_{i-1})$,

$$\begin{aligned} &= \sum_{(s_0 a_0, \dots, s_t a_t) \in S'^t} \left(\bigodot_{i=0}^t \phi_1(s_i a_i) \right) \cdot \prod_{i=1}^t \mathbf{P}[s_{i-1} a_{i-1}, s_i a_i] \\ &= (M\phi_1 V_{\odot}^{\leq t} \phi_2)^D(sa). \end{aligned}$$

- Finally, according to the CTML definition in [1] for the final value of ϕ on D, denoted by D_ϕ and asCTML Definition 12 for the final value of ϕ on M, denoted by M_ϕ , we have:

$$\begin{aligned} M_\phi &= \sum_{s \in \mathcal{S}} \left(\sum_{a \in \mathcal{ACT}} \phi(s, a) \cdot \delta(s, a) \right) \cdot \pi_0(s) \\ &= \sum_{(sa) \in S'} \phi(sa) \cdot \pi'_0(sa) \quad \text{by the translation algorithm} \\ &= D_\phi \end{aligned}$$

We now compare the expressiveness between asCTML and CTML. First of all, there are asCTML queries that are not expressible by CTML; see Sect. 5 for

example queries 1, 2, and 5. On the other hand, although CTML cannot be directly expressed by asCTML, any CTML formula can be simulated by a corresponding asCTML formula. Algorithm 1 gives details about the simulation. The overall idea about the simulation is that, for each case of CTML formula, we set the value of atomic formulas such that $f(s, a) = f(s)$, then compute the corresponding asCTML formula, then sum over the actions of the computed asCTML formula. The correctness of this algorithm is proven. Due to the limited space, the proof is omitted here.

We are ready to discuss the expressiveness between asCTML and the established probabilistic model checker (PRISM) [12, 13]. In general, asCTML and PRISM are incomparable. First of all, PRISM is a comprehensive tool that incorporates PCTL, CSL, PLTL, and PCTL*, which means there are a lot of queries that can be expressed by PRISM cannot be expressed by asCTML. For example, PLTL formulas that have LTL formula of the form $(a_1 U b_1) \wedge \dots \wedge (a_n U b_n)$ (where $a_i, b_i, 1 \leq i \leq n$ are atomic propositions [14]) are not expressible by asCTML or CTML, because those are proven to be NP-hard [15], whereas the overall computational complexity of both asCTML and CTML are linear in the size of the operators in a formula, and polynomial in the size of a model. In addition, PRISM has mechanisms to associate a real-valued rewards to actions and/or states on DTMC models. It has $R=?$ and $P=?$ operators that denote rewards and probability, respectively, and can express formulas such as $R=?[\cdot]$ or $P=?[\cdot]$, where $[\cdot]$ denotes a classic temporal logic path formula (see examples in [16]). Nonetheless, the core formalisms in PRISM still defines a satisfaction relation; in another words, they cannot express formulas such as $R=?[f_1 U [R=?f_2 U f_3]]$, or $R=?[f_1 U [P=?f_2 U f_3]]$ (where f_1, f_2, f_3 are atomic formulas associated with actions and/or states), because unlike asCTML or CTML they cannot nest real values naturally in general (see the most recent PRISM work announced here: <http://www.prismmodelchecker.org/>).

5 The University Graduation Application

We have a prototype software tool implemented in Java, for the evaluation of asCTML and CTML queries. The heart of the software tool is the CTML engine that implements all the CTML algorithms discussed in [1]. For asCTML queries, we add an additional translator that translates from MAMC to DTMC and asCTML to CTML as discussed in Sect. 4. The software has been tested on several applications, including large dining philosopher models with millions of states and edges. Due to the limited space, for this work, we select to present a small model structure to illustrate capabilities of asCTML and CTML. With this small structure, we tend to show users more clearly the quantification of action driven paths, how action dependent queries can be expressed by asCTML but not CTML, how action independent queries can be expressed by CTML, and their syntactic similarities. More importantly, through this small application example, we tend to deliver the overall specification idea of asCTML with respect to CTML.

Algorithm 1. *simulate*(ϕ' , M)

Input: ϕ', M **Output:** vector π such that $\pi[s] = \phi'(s)$

```

switch  $\phi'$  do
  case  $f'$ 
    for  $\forall s \in \mathcal{S}$  do
       $\pi[s] = f'(s)$ ;
     $f = \text{setOperand}(\pi, M)$ ;
    for  $\forall s \in \mathcal{S}$  do
       $\pi[s] = \sum_{a \in \mathcal{ACT}} f(s, a) \delta(s, a)$ ;
    return  $\pi$ ;
  case  $\phi'_1 \odot \phi'_2$ 
     $\phi_1 = \text{setOperand}(\text{simulate}(\phi'_1, M), M)$ ;
     $\phi_2 = \text{setOperand}(\text{simulate}(\phi'_2, M), M)$ ;
    for  $\forall s \in \mathcal{S}$  do
       $\pi[s] = \sum_{a \in \mathcal{ACT}} (\phi_1 \odot \phi_2)(s, a) \delta(s, a)$ ;
    return  $\pi$ ;
  case  $\phi'_1 \bowtie \phi'_2$ 
     $\phi_1 = \text{setOperand}(\text{simulate}(\phi'_1, M), M)$ ;
     $\phi_2 = \text{setOperand}(\text{simulate}(\phi'_2, M), M)$ ;
    for  $\forall s \in \mathcal{S}$  do
       $\pi[s] = \sum_{a \in \mathcal{ACT}} (\phi_1 \bowtie \phi_2)(s, a) \delta(s, a)$ ;
    return  $\pi$ ;
  case  $1 - \phi'$ 
     $\phi = \text{setOperand}(\text{simulate}(\phi', M), M)$ ;
    for  $\forall s \in \mathcal{S}$  do
       $\pi[s] = \sum_{a \in \mathcal{ACT}} (1 - \phi)(s, a) \delta(s, a)$ ;
    return  $\pi$ ;
  case  $MX\phi'$ 
     $\phi = \text{setOperand}(\text{simulate}(\phi', M), M)$ ;
    for  $\forall s \in \mathcal{S}$  do
       $\pi[s] = \sum_{a \in \mathcal{ACT}} (MX\phi)(s, a) \delta(s, a)$ ;
    return  $\pi$ ;
  case  $M\phi'_1 U_{\odot}^{\leq t} \phi'_2$ 
     $\phi_1 = \text{setOperand}(\text{simulate}(\phi'_1, M), M)$ ;
     $\phi_2 = \text{setOperand}(\text{simulate}(\phi'_2, M), M)$ ;
    for  $\forall s \in \mathcal{S}$  do
       $\pi[s] = \sum_{a \in \mathcal{ACT}} (M\phi_1 U_{\odot}^{\leq t} \phi_2)(s, a) \delta(s, a)$ ;
    return  $\pi$ ;
  case  $M\phi'_1 V_{\odot}^{\leq t} \phi'_2$ 
     $\phi_1 = \text{setOperand}(\text{simulate}(\phi'_1, M), M)$ ;
     $\phi_2 = \text{setOperand}(\text{simulate}(\phi'_2, M), M)$ ;
    for  $\forall s \in \mathcal{S}$  do
       $\pi[s] = \sum_{a \in \mathcal{ACT}} (M\phi_1 V_{\odot}^{\leq t} \phi_2)(s, a) \delta(s, a)$ ;
    return  $\pi$ ;

```

Algorithm 2. *setOperand*(π, M)**Input:** vector π, M **Output:** asCTML atomic formula f such that $f(s, a) = \pi[s]$

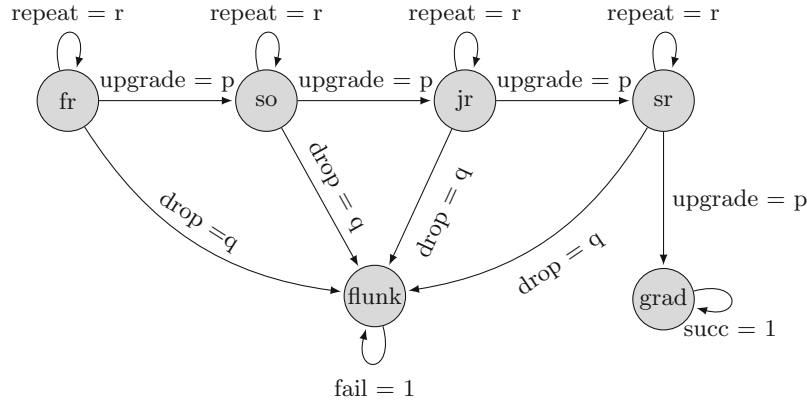
```

for  $\forall s \in \mathcal{S}$  do
  for  $\forall a \in \mathcal{ACT}$  do
     $f(s, a) = \pi[s]$ ;
return  $f$ ;

```

5.1 Specification Examples

The *University Graduation* example as shown in Fig. 2 is from [17]. Each year, at a fictitious four year undergraduate university, a student has probability p to move up to a higher grade, has probability r to repeat for the same grade, and probability q to drop the school. The following presents interesting queries for this model.

**Fig. 2.** University graduation example

1. “What’s the average number of years of repeating at the freshman level?”
First of all, the query requires to count the **repeat** action at the freshman year only. So we define an atomic formula *repeats-fr* such that it has value 1 on $(fr, repeat)$, and 0 on all others. Also, for this model, there are only two destinations which are either finish successfully indicated by **grad** or **flunk**. So we define another atomic formula *finish* such that it has value 1 on $(grad, succ)$ and $(flunk, fail)$, and 0 otherwise. The query can then be expressed as

$$M \text{ repeats-fr } U_+ \text{ finish}$$

In general, this type of query is not expressible by CTML, especially if we change the value of $repeats-fr(fr, repeat)$ to some other positive value other than 1. In this particular case, however, we can use the CTML expression $M fr U_+(1 - fr)$ to get the average number of years spent as a freshman, and then we subtract one to get the number of repeats.

2. “What’s the average number of repeating at all levels of grades given that students graduate successfully?” Unlike the previous query, this query asks for the quantification of the repeating behavior at all levels of grades, so we define an atomic formula $repeats$ such that $repeats(\{fr, so, jr, sr\}, repeat) = 1$ and 0 for all others. Also, we need an atomic formula $graduate(grad, succ) = 1$ and 0 otherwise. Then the query can be expressed as

$$M repeats U_+ graduate$$

By the definition of the conditional expectation equation [18]:

$$E(B \mid A) = \sum_{\forall b} b \cdot \Pr(B = b \mid A) = \frac{\sum_{\forall b} b \cdot \Pr(B = b \cap A)}{\Pr(A)} \quad (4)$$

the result of the expression must be divided by the quantity of $M one U_{\times} graduate$. The query is not expressible by CTML in general.

3. “What’s the probability to quit the school after sophomore year?” Let $quit-after-soph$ be an atomic formula such that $quit-after-soph(\{so, jr, sr\}, drop) = 1$, and 0 for all others. Then the query can be expressed as

$$M one U_{\times} quit-after-soph$$

This query is expressible by CTML as

$$M one U_{\times} (M so U_{\times} flunk)$$

and by PLTL as

$$P_{?} = F(so U flunk)$$

In this case, asCTML requires one path operator $U_{\times}$, whereas both CTML and PLTL requires two path operators.

4. “What’s the probability of graduation in 6 years?” Let $graduate$ be an atomic formula with $graduate(grad, succ) = 1$ and 0 otherwise. Then the query can be expressed as

$$M one U_{\times}^{\leq 6} graduate$$

This query is expressible by CTML, with the same syntax as for asCTML, except that with CTML the atomic formulas $graduate$ and one quantify the corresponding states only, rather than the (state, action) pairs.

5. “Given that a student reached the junior state and never repeated a year before, what’s the probability that the student will graduate in 3 years?” This query requires to only quantify the *upgrade* action at the *fr* and *so* states as the conditional statement imposes. Let $no-repeat-soph(\{fr, so\}, upgrade) = 1$, and 0 otherwise. Let $graduate(grad, succ) = 1$ and 0 otherwise. Let $junior(jr, \{repeat, upgrade, drop\}) = 1$, and 0 otherwise. Then this query can be expressed as

$$M no-repeat-soph U_{\times}(junior \times M one U_{\times}^{\leq 3} graduate)$$

and the result must be divided by $M no-repeat-soph U_{\times} junior$, according to the conditional probability definition [18]. Since the query depends on actions, it is not expressible by CTML.

Note that in case there are multiple actions between the same pair of states, the idea of the specification would be the same.

5.2 Experimental Results

We now discuss experimental results for the queries. First, we come up with a MAMC model structure for the University Graduation example, and a set of atomic state+action formulas for the queries. we (arbitrary) set the probabilities p, r, q to be 0.1, 0.8, and 0.1, respectively. The MAMC model and the atomic state+action formulas are then translated into DTMC model and CTML atomic state formulas, which are then plugged into CTML software for evaluation of each query. Since the specifications are syntactically the same for asCTML and CTML, they can either be included in MAMC model and carried over to the DTMC model, or be put in the translated DTMC model directly. For this small model, we omit the discussion of testing environment and CPU time. Table 1 presents the numerical results for each query discussed above. All outputs are verified by the numerical solutions computed by hand.

Table 1. Numerical results of asCTML queries for Fig. 2

#	Query	Initial state	Numerical result
1	$M repeats-fr U_{+} finish$	fr	0.11111
2	$\frac{M repeats U_{+} graduate}{M one U_{\times} graduate}$	fr	0.44443
3	$M one U_{\times} quit-after-soph$	fr	0.26459
4	$M one U_{\times}^{\leq 6} graduate$	fr	0.57344
5	$\frac{M no-repeat-soph U_{\times}(junior \cdot M one U_{\times}^{\leq 3} graduate)}{M no-repeat-soph U_{\times} junior}$	fr	0.64000

6 Related Work

There is a body of past work on stochastic model checking formalisms. The most relevant work such as PRISM and CTML has already been discussed in Sect. 4. Other rewards augmented stochastic formalisms include PRCTL [19] and CSRL [20] (which is similar to PRCTL, but operates on continuous-time Markov chains). They are more related to asCTML’s prior work, CTML, since all those are the extensions of PCTL with rewards functions, but they don’t have action formulas. For action and state based stochastic formalisms, CSL^{TA} [10] and asCSL [5] (which is an extension of CSL [21] and aCSL [22] that work with continuous-time Markov chains) are closely related to asCTML in a sense that both allow multiple actions between a pair of states and can quantify paths featured by a combination of states and actions. CSL^{TA} is more expressive than asCSL for reasoning state and action properties over probabilistic systems. Unlike asCTML, however, they don’t support real-valued rewards. Another work we would like to mention is *performance trees* [23], as it can describe results of various types (real-valued or otherwise, including distributions). Unlike CTML or asCTML, however, performance trees is a more general framework that relies on the existing performance evaluation algorithms as well as some of the existing model checking algorithms (such as PCTL, CSL, etc.) for the expression of both logic and real-valued measures.

7 Conclusions and Future Work

We have introduced a formal query language, asCTML, for describing quantitative performance measures. The combination of nesting and real-valued state/action formulas, particularly the nesting of action formulas, unique to asCTML, extend the expressive power of the familiar temporal operators. Through proper translation, asCTML can simulate any CTML formula. Our asCTML algorithm has complexity similar to that of PCTL, but better than PLTL. We are currently working on to apply asCTML for continuous time Markov chains or semi-Markov processes. For unbounded until and unbounded weak until queries, this is fairly straightforward: since asCTML can handle real-valued state/action formulas, we can instead analyze the embedded DTMC using asCTML, and scale the state/action formulas by the expected time spent in each state or action. The time bounded versions of these formulas are not so straightforward to handle, and will likely require significant changes.

References

1. Miner, A.S., Jing, Y.: A formal language toward the unification of model checking and performance evaluation. In: Al-Begain, K., Fiems, D., Knottenbelt, W.J. (eds.) *ASMTA 2010. LNCS*, vol. 6148, pp. 130–144. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-13568-2_10
2. Ciardo, G., Trivedi, K.S.: A decomposition approach for stochastic reward net models. *Perform. Eval.* **18**, 37–59 (1993)
3. Hansson, H., Jonsson, B.: A logic for reasoning about time and reliability. *Formal Aspects Comput.* **6**(5), 512–535 (1994)
4. Courcoubetis, C., Yannakakis, M.: The complexity of probabilistic verification. *J. ACM* **42**(4), 857–907 (1995)
5. Baier, C., Cloth, L., Haverkort, B.R., Kuntz, M., Siegle, M.: Model checking Markov chains with actions and state labels. *IEEE Trans. Softw. Eng.* **33**, 209–224 (2007)
6. Dijkstra, E.W.: Hierarchical ordering of sequential processes. *Acta Informatica* **1**, 115–138 (1971)
7. Obal, W.D., Sanders, W.H.: State-space support for path-based reward variables. *Perform. Eval.* **35**(3–4), 233–251 (1999)
8. Stewart, W.J.: *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press, Princeton (1994)
9. Cohn, D.L.: *Measure Theory*. Birkhäuser, Boston (1980)
10. Donatelli, S., Haddad, S., Sproston, J.: Model checking timed and stochastic properties with CSL^{TA} . *IEEE Trans. Softw. Eng.* **35**(2), 224–240 (2009)
11. Clark, G., Gilmore, S., Hillston, J.: Specifying performance measures for PEPA. In: Katoen, J.-P. (ed.) *ARTS 1999. LNCS*, vol. 1601, pp. 211–227. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48778-6_13
12. Kwiatkowska, M.: Quantitative verification: models, techniques and tools. In: *Proceedings of the 6th joint meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering*, New York, NY, USA, pp. 449–458. ACM (2007)
13. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: verification of probabilistic real-time systems. In: Gopalakrishnan, G., Qadeer, S. (eds.) *CAV 2011. LNCS*, vol. 6806, pp. 585–591. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22110-1_47
14. Pnueli, A.: The temporal semantics of concurrent programs. *Theor. Comput. Sci.* **13**, 45–60 (1981)
15. Schnoebelen, P.: The complexity of temporal logic model checking. *Adv. Modal Log.* **4**, 393–436 (2002)
16. Kwiatkowska, M., Norman, G., Parker, D.: Probabilistic verification of Herman’s self-stabilisation algorithm. *Formal Aspects Comput.* **24**(4–6), 661–670 (2012)
17. Kemeny, J.G., Snell, J.L.: *Finite Markov Chains*. D. Van Nostrand Company Inc., Princeton (1960)
18. Pitman, J.: *Probability*. Springer, New York (1993). <https://doi.org/10.1007/978-1-4612-4374-8>
19. Andova, S., Hermanns, H., Katoen, J.P.: Discrete-time rewards model-checked. In: Larsen, K.G., Niebert, P. (eds.) *Formal Modeling and Analysis of Timed Systems*, pp. 88–104. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-40903-8_8

20. Baier, C., Haverkort, B., Hermanns, H., Katoen, J.-P.: On the logical characterisation of performability properties. In: Montanari, U., Rolim, J.D.P., Welzl, E. (eds.) ICALP 2000. LNCS, vol. 1853, pp. 780–792. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-45022-X_65
21. Baier, C., Haverkort, B., Hermanns, H., Katoen, J.P.: Model-checking algorithms for continuous-time Markov chains. *IEEE Trans. Softw. Eng.* **29**(6), 524–541 (2003)
22. Hermanns, H., Katoen, J.-P., Meyer-Kayser, J., Siegle, M.: Towards model checking stochastic process algebra. In: Grieskamp, W., Santen, T., Stoddart, B. (eds.) IFM 2000. LNCS, vol. 1945, pp. 420–439. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-40911-4_24
23. Suto, T., Bradley, J.T., Knottenbelt, W.J.: Performance trees: expressiveness and quantitative semantics. In: Proceedings of the 4th International Conference on Quantitative Evaluation of Systems, Washington DC, USA, pp. 41–50. IEEE Computer Society (2007)